

Eager Loading ,Lazy Loading and explicit Loading in Entity Framework Core, including examples, sample entities, sample data, and how EF loads related data in each case.

1. What Are Navigation Property Loading Strategies?

In EF Core, when you query an entity that has relationships (One-to-Many, Many-to-Many, etc.), EF must decide **when** and **how** to load the related data.

There are three strategies:

1. **Eager Loading** – Load related data immediately with the main entity.
2. **Lazy Loading** – Load related data only when it is accessed.
3. **Explicit Loading** – Load related data manually by calling `.Load()`.

Your question focuses on **Eager** and **Lazy** loading.

2. Sample Entity Model

Consider two tables: **Department** (1) and **Employee** (Many)

```
public class Department
```

```
{  
    public int DepartmentId { get; set; }  
    public string Name { get; set; }  
  
    public ICollection<Employee> Employees { get; set; }  
}
```

```
public class Employee
```

```
{  
    public int EmployeeId { get; set; }  
    public string FullName { get; set; }  
    public int DepartmentId { get; set; }
```

```
public Department Department { get; set; }  
}
```

3. Sample Data

Assume the database contains:

Departments Table

DepartmentId	Name
--------------	------

1	IT
2	Accounts

Employees Table

EmployeeId	FullName	DepartmentId
------------	----------	--------------

101	Ravi Kumar	1
102	Meena Sharma	1
201	Arjun Patel	2

4. EAGER LOADING

4.1 Concept

Eager loading fetches **main entity + related entities** in a **single query** sent to the database.

Use eager loading when:

- You know you need related data.
- You want to reduce the number of database calls.

EF Core uses the **Include()** and **ThenInclude()** methods.

4.2 Example – Load Department with Employees

```
var departments = await _context.Departments
    .Include(d => d.Employees)
    .ToListAsync();
```

What happens internally?

EF executes **one SQL query**:

```
SELECT d.DepartmentId, d.Name,
       e.EmployeeId, e.FullName, e.DepartmentId
FROM Departments d
LEFT JOIN Employees e ON d.DepartmentId = e.DepartmentId;
```

Output (in memory)

- Department: IT
 - Employees: Ravi Kumar, Meena Sharma
- Department: Accounts
 - Employees: Arjun Patel

Key Characteristics of Eager Loading

- Fewer queries (usually 1).
 - Slightly heavier initial load because more data is loaded upfront.
 - Excellent when related data is definitely required.
-

5. LAZY LOADING

5.1 Concept

Lazy loading fetches related data **only when the navigation property is accessed in code**.

EF sends **multiple SQL queries**, loading data on demand.

To enable lazy loading in EF Core:

1. Install: Microsoft.EntityFrameworkCore.Proxies
2. Configure:

```
services.AddDbContext<AppDbContext>(options =>
    options.UseLazyLoadingProxies()
        .UseSqlServer(connectionString));
```

3. Mark navigation properties as virtual:

```
public virtual ICollection<Employee> Employees { get; set; }
```

5.2 Example – Lazy Loading in action

```
var department = await _context.Departments
    .FirstAsync(d => d.DepartmentId == 1);
```

```
// Employees not loaded yet
```

```
var employees = department.Employees; // Lazy load happens here
```

What happens internally?

EF executes **two queries**:

1. Query for Department:

```
SELECT * FROM Departments WHERE DepartmentId = 1;
```

2. When Employees is accessed:

```
SELECT * FROM Employees WHERE DepartmentId = 1;
```

Output

```
department.Name = "IT"
```

```
department.Employees → loads and returns [Ravi, Meena]
```

Key Characteristics of Lazy Loading

- Fast initial load (loads only main entity first).
- Additional queries later (may cause N+1 problems).
- Ideal when related data is optional or rarely used.

6. COMPARISON TABLE

Feature	Eager Loading	Lazy Loading
When related data is loaded	Immediately	When accessed
Database calls	Usually 1	Multiple
Performance	Good when related data required	Good when related data optional
Code	Uses Include()	Uses proxies + virtual navigation
Suitable for	API responses, data display	Large graphs, conditional loading

7. Practical Usage Guidance

Use Eager Loading when:

- You are building an API returning DTOs including child objects.
- You know you need related data every time.
- You want predictable SQL queries.

Use Lazy Loading when:

- You want flexibility.
- You are working with large object graphs.
- You want to avoid heavy initial loading.

What Is Explicit Loading?

Explicit Loading means:

You load the main entity first,
and then load its related data **manually**, by calling `.LoadAsync()`.

EF Core does *not* automatically load navigation properties.
You control exactly **when** and **what** to load.

Explicit loading is useful when:

- You do not want Lazy Loading
- You want control over performance
- You want to avoid loading too much data
- You do not want automatic loading triggered by JSON serialization

EXPLICIT LOADING Example

Step 1: Load the Department

```
var department = await _context.Departments  
    .FirstOrDefaultAsync(d => d.DepartmentId == 1);
```

At this point:

- **Department** is loaded
- **Employees are NOT loaded**

Step 2: Explicitly load Employees (One-to-Many)

```
await _context.Entry(department)  
    .Collection(d => d.Employees)  
    .LoadAsync();
```

Now the Employees collection is loaded.